



Мультиотенантная платформа для AI-организаций

Реализовано: ASRP · **Роль:** Основная разработка / контрибьютор №1 (бэкенд, админ-панель, слой развёртывания) · **Тип сотрудничества:** Контракт · **Домен:** Мультиотенантная SaaS для бизнес-ИИ-агентов

1. Краткое резюме

Мультиотенантная SaaS-платформа для создания и работы «ИИ-организаций»: админ-панель, в которой оператор бизнеса описывает ИИ-агента (его назначение, инструкции, инструменты и базу знаний), а затем разворачивает этого же агента сразу в нескольких клиентских каналах — встраиваемом чат-виджете для сайта, надстройке для Outlook и автономном обработчике почты. Оператору не нужно писать код: платформа превращает описание задачи на естественном языке в настроенного, готового к развёртыванию агента.

Архитектурно это две взаимодействующие системы внутри одного общего монорепозитория: **TypeScript SaaS** (бэкенд на NestJS + админ-панель на React + два клиентских интерфейса) — источник истины по тенантам, пользователям, агентам, развёртываниям и данным, — и **Python-сервис авторинга/исполнения агентов**, генерирующий агентов из текста и запускающий их. SaaS — быстрый транзакционный реестр; Python-сервис — медленный, пульсирующий уровень генерации/исполнения. Рассуждения агентов выполняются на **Gemini от Google (через Vertex AI)** и **Claude от Anthropic (через LiteLLM)**; окружающая инфраструктура — облачная (управляемый поиск для retrieval/RAG, шина сообщений для асинхронной работы, blob-хранилище для документов, управляемый email-сервис и хостинг контейнеров/статик в регионе EC). Платформа развёрнута в продакшне и в активной разработке.

2. Позиционирование и проблема

Тезис продукта: **малому и среднему бизнесу нужны ИИ-агенты, выполняющие реальную бэк-офисную работу — сортировку почты, чтение документов, ответы посетителям сайта, — но они не могут (и не должны быть обязаны) содержать ML-команду.** Ответ — панель управления (control plane), обращённая к оператору. Нетехнический администратор описывает задачу прозой, платформа собирает рабочего агента, и одним кликом разворачивает его туда, куда приходит работа.

До этой платформы развёртывание такого агента означало кастомный prompt engineering, отдельную интеграцию под каждый канал и самодельный слой тенантов/прав для каждого клиента. Цель — **единый продукт**, где определение агента, приём знаний, подключение инструментов, мультиотенантная изоляция и мультиканальное развёртывание — полноценные переиспользуемые примитивы, так что подключение нового бизнеса становится настройкой, а не отдельным проектом.

Этот кейс — **история оператора/SaaS**; агент-разработческая инфраструктура, на которой он стоит, — отдельный объём работ под NDA.

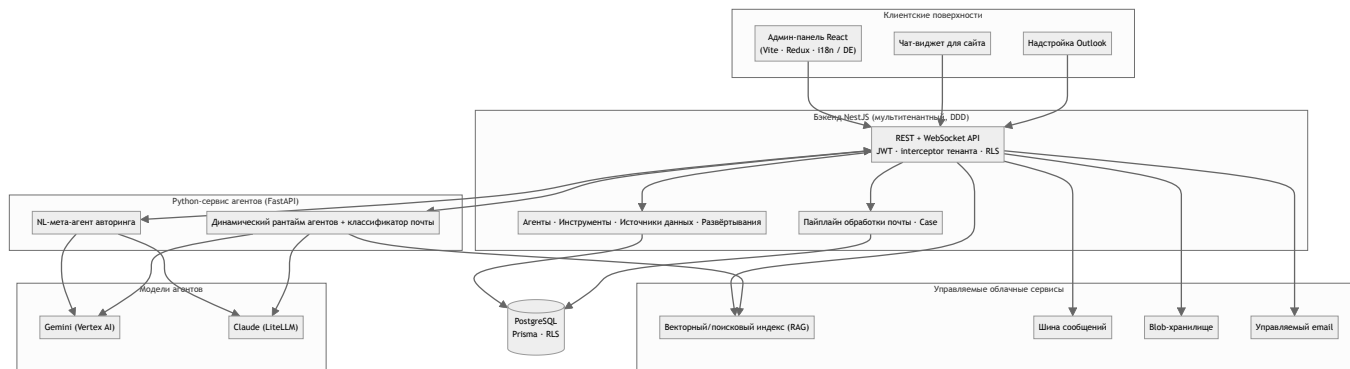
3. Что было реализовано

Реализованная поверхность продукта — от слоя изоляции до повседневной эксплуатации:

- **Мультитенантный control plane** — тенанты, пользователи только по инвайтам, роли/статусы и изоляция данных по тенанту на уровне базы данных; контекст тенанта передаётся с каждым запросом из JWT.
- **Конструктор агентов** — пошаговый визард (профиль → источники данных → конфигурация) и рабочее пространство деталей (промпт, конфигурация, шаблоны писем/экранов, расписания, суб-агенты, связи), а также живая песочница **Test**, запускающая агента через Python-сервис — всё в рамках одного тенанта.
- **Авторинг агента на естественном языке** — Python-мета-агент, который разбирает текстовое ТЗ и делегирует специализированным суб-мейкерам (конфигурация, инструкции, генерация экранов и email-шаблонов) вплоть до полной провалидированной конфигурации, которую SaaS может развернуть.
- **Знания / RAG** — файловые источники данных, загружаемые и индексируемые в управляемый векторный/поисковый индекс (каждый чанк помечен тенантом), подключаемые к любому агенту для ответов на основе фактов.
- **Три канала развёртывания из одного агента** — встраиваемый чат-виджет для сайта (с полноценным конструктором брендинга по тенантам), надстройка Outlook и автономный почтовый агент, который классифицирует входящую почту относительно собственных агентов тенанта и маршрутизирует каждое сообщение.
- **Пайплайн обработки почты** — обработка входящих писем с отслеживанием состояния и повторами (`PENDING` → `PROCESSING` → `PROCESSED` / `FAILED` / `MAX_RETRIES_EXCEEDED`), с динамической классификацией по тенанту и независимой от источника моделью **Case**.
- **Интеграция инструментов и секреты** — типизированный реестр инструментов с зашифрованными секретами на уровне агент/тенант, а также учётные данные почтовых подключений Google/Microsoft.
- **Эксплуатационная поверхность** — журналы активности/аудита, журналы исполнения и трассировки по шагам, планирование на cron, WebSocket-шлюзы для живых обновлений, хранение логов и API, документированный через Swagger.

4. Архитектура

Основа: **админ-панель React** → **API на NestJS (в рамках тенанта, со слоями DDD)** → **PostgreSQL (Prisma)**, при этом бэкэнд NestJS оркестрирует управляемые облачные сервисы и делегирует авторинг/исполнение агентов **Python-сервису**. Одни и те же определения агентов расходятся на три поверхности развёртывания.



Компонент	Зона ответственности
React admin	UI оператора: сборка агентов, управление тенантами/ пользователями, настройка развёртываний, логи/ аналитика. Vite + Redux Toolkit + Tailwind + react-i18next (немецкий; готовность к i18n), живые обновления по Socket.IO.
Бэкенд NestJS	Авторитетный API. Слои DDD (domain / application / infrastructure), auth JWT/Passport, контекст тенанта на запрос, Prisma над PostgreSQL с RLS, Swagger, cron, WebSocket-шлюзы.
Python-сервис агентов	Авторинг + исполнение агентов: NL-мета-агент, динамический рантайм, эндпоинты run/chat и пайплайн обработки почты.
Чат-виджет	Встраиваемая клиентская поверхность по тенантам — ладер, монтирующий iframe и общающийся с бэкендом по Socket.IO.
Настройка Outlook	Приносит агента в почтовый ящик как Office.js task-pane add-in.
Модели агентов	Gemini через Google Vertex AI и Claude через LiteLLM, за allow-list моделей (без «тихого» дефолта).
Управляемое облако	Векторный/поисковый индекс (retrieval), шина сообщений (async), blob-хранилище (документы), управляемый email (отправка), хостинг контейнеров/ статики, идентити Microsoft.

5. Инженерные разборы

- **Изоляция тенантов на уровне БД.** Row-level security (FORCE RLS) плюс контекст тенанта из JWT на каждый запрос — изоляцию обеспечивает база данных, а не только код приложения: ошибка в запросе не «протечёт» между тенантами.
- **Одно определение, три рантайма.** Агент описывается один раз; виджет, надстройка Outlook и обработчик почты исполняют одно и то же определение через Python-сервис. Добавление канала не форкает модель агента.

- **Естественный язык → настроенный агент.** Мета-агент авторинга разбирает текстовое ТЗ и делегирует специализированным суб-мейкерам (конфигурация, инструкции, экраны и email-шаблоны), возвращая провалидированную конфигурацию — превращая «опишите задачу» в готового агента примерно за минуту.
- **Почта с состоянием и повторами.** Входящий пайплайн — явная машина состояний с ограниченными ретраями и независимой от источника моделью Case, поэтому обработка почты наблюдаема и восстанавливаема.
- **Allow-list моделей.** Рассуждения — на Gemini и Claude за явным allow-list: без «тихого» дефолта, так что стоимость и поведение предсказуемы по тенанту.

6. Технологический стек

Слой	Выбор
Фронтенд админки	React + Vite, Redux Toolkit, Tailwind, react-i18next, Socket.IO
Бэкенд	NestJS (слои DDD), JWT/Passport, Prisma, Swagger, WebSocket-шлюзы, cron
База данных	PostgreSQL с row-level security (FORCE RLS)
Сервис агентов	Python + FastAPI — NL-мета-агент авторинга, динамический рантайм, пайплайн почты
Модели	Gemini (Vertex AI) + Claude (LiteLLM) за allow-list моделей
Облако	Управляемый вектор/поиск (RAG), шина сообщений, blob-хранилище, управляемый email, хостинг контейнеров/статика (регион EC)

7. Данные и интерфейсы

Ключевые сущности — **Tenant, User, Agent, DataSource, Deployment, Tool и Case**, все в рамках тенанта в PostgreSQL под row-level security. Агенты несут промпт/конфигурацию/шаблоны/расписания и могут связываться с суб-агентами. Бэкенд публикует REST API (Swagger) и WebSocket-шлюзы для живых обновлений; Python-сервис публикует эндпоинты run/chat, которые SaaS вызывает для авторинга и исполнения. Входящая почта проходит через модель Case с явной машиной состояний.

8. Надёжность и эксплуатация

- **Изоляция на уровне БД (FORCE RLS)** — защита в глубину сверх проверок приложения.
- **Наблюдаемость** — журналы активности/аудита, исполнения, трассировки по шагам, хранение логов.
- **Асинхронная и плановая работа** — шина сообщений для пульсирующей генерации/исполнения и расписания на cron.
- **Процесс релиза** — путь staging → production (владелец — ASRP), Python-сервис масштабируется независимо от транзакционного SaaS.

9. Метрики

Метрика	Значение
Каналов развёртывания на агента	3 (web / Outlook / email)
Авторинг агента из прозы	~1 мин (цель)
Вклад (оба ключевых репозитория)	№1, ~45% коммитов
Изоляция тенантов	PostgreSQL FORCE RLS + JWT-контекст тенанта

10. Статус и планы

Статус: продакшн, активная разработка. Платформа — хостируемый мультитенантный SaaS (только по инвайтам). Направление развития: более широкое покрытие каналов, более богатый авторинг и глубокая аналитика. Агент-разработческая инфраструктура под ней (SDK, рантайм, шаблоны агентов) — отдельный объём работ под NDA.

11. Что реализовал ASRP

ASRP была контрибьютором №1 (~45% коммитов) в обоих ключевых репозиториях: мультитенантный бэкенд на NestJS (изоляция тенантов, агенты/инструменты/источники данных/развёртывания, пайплайн обработки почты), админ-панель на React и слой развёртывания агентов, разводящий одно определение в виджет, надстройку Outlook и обработчик почты. ASRP также владел процессом релиза staging → production.

12. try it / Посмотреть / попробовать

Платформа — хостируемый мультитенантный SaaS (только по инвайтам), поэтому анонимного публичного входа нет, а исходные репозитории приватны. Клиент здесь не называется; идентифицирующие детали доступны под NDA. Скриншоты не приводятся — все снимки из источника содержали реальные хостнеймы инфраструктуры и данные тенантов.

13. FAQ (для разговоров с клиентами)

- **Это одно- или мультитенантная система?** Мультитенантная, изоляция обеспечивается на уровне базы данных через row-level security и контекст тенанта из JWT на каждый запрос.
- **Нужно ли операторам программировать?** Нет — агент описывается на естественном языке, а платформа собирает провалидированную, готовую к развёртыванию конфигурацию.
- **Как один агент используется в разных каналах?** Агент описывается один раз и исполняется через Python-сервис с трёх поверхностей: виджет сайта, надстройка Outlook и обработчик почты.
- **Какие модели используются?** Gemini от Google (через Vertex AI) и Claude от Anthropic (через LiteLLM) за явным allow-list моделей.

- **Почему анонимизировано?** Клиент под NDA. Здесь показаны детали уровня возможностей; идентифицирующая конкретика — по запросу под NDA.
-

Анонимизированный кейс клиентского проекта, подготовленный ASRP для портфолио. Клиент под NDA и не называется; ссылки не приводятся. Идентификаторы реализации, эндпоинты, учётные данные и данные тенантов намеренно исключены; приведены только детали уровня возможностей.